ROAD MAP OF THE PROPOSED SYSTEM

MacroSight™ – Macroeconomic-Driven Sales Intelligence Platform

Backend Logical Layering

macrosight/

├── **core/** # System configuration

├── **domain/** # Pure business logic (no Django dependencies)

├── **models/** # Django ORM models

├── **services/** # Forecasting, ingestion, simulation logic

├----**api/** # DRF views, serializers, routers

├── **automation/** # Cron-triggered jobs

├── **governance/** # Model runs, audit, reproducibility

├── **integrations/** # External indicators & policy data

└── **tests/**

Core Domain Objects

- **Sale** (immutable historical fact)
- **Product**
- **TimePeriod**
- **ForecastMetric** (REVENUE | UNITS)
- **ForecastHorizon**
- **Forecast** (aggregate root)
- **ForecastModel** (strategy abstraction)
- **TrainedModel**
- **ModelRun** (governance & audit)
- **ExternalIndicator**

  (macro variables)

Key Enforcement Rule:

- Defined metric
- Explicit time period
- Confidence interval

MACROSIGHT ROADMAP

- Model run metadata

Forecasting Lifecycle

- Accept trained model artifacts from Colab
- Register model metadata (name, horizon, training window)
- Execute predictions via backend services
- Persist forecasts with confidence bounds

Macroeconomic Integration

- Store indicators (inflation, GDP, FX, fuel, tax signals)
- Bind indicators to time periods
- Inject indicators into forecast services

Policy & Legislation Modeling

- Encode tax policy events
- Assign qualitative impact scores
- Link policies to forecast scenarios

6.5 Scenario Simulation

- Clone baseline forecasts
- Apply macro/policy deltas
- Compare forecast outputs side-by-side

6.6 Governance & Audit

- Track every model run
- Store accuracy scores
- Guarantee reproducibility
- Support regulatory and executive trust

. API Design (Backend-First)

7.1 API Standards

- RESTful JSON
- Versioned (/api/v1/)
- Read-optimized endpoints
- Stateless authentication

7.2 Core API Modules

MACROSIGHT ROADMAP

Module	Example Endpoints
Auth	/auth/login, /auth/refresh
Sales	/sales/upload, /sales/query
Products	/products/
Indicators	/indicators/
Forecasts	/forecasts/generate, /forecasts/results
Scenarios	/scenarios/run, /scenarios/compare
Governance	/models/runs, /models/audit
Health	/health/

---

8. ML Training Strategy (Google Colab)

Why Colab

- Cost-effective GPU/CPU
- Data science flexibility
- Rapid experimentation

Workflow

1. Backend exports training data (CSV/Parquet)
2. Model training executed in Colab
3. Trained model artifact produced (pickle / joblib / ONNX)
4. Artifact uploaded to backend
5. Backend registers model & metadata
6. Predictions served via APIs

Important:
Colab **never** directly accesses production DB.

---

9. Automation & Scheduling

Task	Mechanism
Forecast refresh	Cron jobs
Indicator updates	Scheduled ingestion

MACROSIGHT ROADMAP

Task	Mechanism
Model health checks	Periodic runs
Cleanup	Background jobs

Automation is **backend-only**, container-agnostic.

---

10. Security Model

- Backend-enforced validation
- Role-based access control
- JWT-based authentication
- No trust placed on frontend inputs
- Secrets managed via environment variables

---

11. Non-Functional Guarantees

Area	Guarantee
Performance	Predictable API response times
Reliability	Reproducible forecasts
Scalability	Horizontal scaling ready
Maintainability	Clear domain-service separation
Longevity	UI can be replaced without backend rewrite

Model learning and framing

1. Core Requirement Clarification (Non-Rule-Based Learning)

MacroSight™ must use data-driven learning models, not hardcoded logic.
This implies:

- No if/else business rules for prediction
- Parameters learned from data
- Ability to update behavior as new data arrives
- Support for multivariate time series (sales + macro indicators)
- Backend-controlled training, inference, and governance

MACROSIGHT ROADMAP

This excludes:

- Pure statistical rules
- Spreadsheet-style forecasting
- Manually tuned coefficients
- **Primary ML Paradigm: Multivariate Time Series Learning**

- MacroSight™ operates on **temporal data with exogenous variables**.
- **Mathematical Framing**
- Y(t) = f(
- past sales,
- macro indicators,
- policy events,
- seasonality,
- external shocks
-)
- Where f is **learned**, not programmed.

andatory ML Techniques (Production-Grade)

3.1 Gradient Boosted Decision Trees (GBDT)

Examples

- XGBoost
- LightGBM
- CatBoost

Why this is mandatory

- Learns non-linear relationships automatically
- Handles mixed data types (numeric, categorical, policy flags)
- Extremely strong on structured business data
- No feature engineering rules required
- Stable and explainable (SHAP)

What it learns

- Impact of inflation, tax changes, FX, fuel
- Interaction effects between indicators
- Non-obvious drivers of sales behavior

Use Case in MacroSight

- Core short- to mid-term forecasting engine

MACROSIGHT ROADMAP

- Default baseline model

Recurrent Neural Networks (LSTM / GRU)

Why

- Learns temporal dependencies automatically
- Remembers long-term patterns
- No explicit lag logic required

Capabilities

- Learns seasonality
- Learns delayed policy effects
- Learns momentum and decay patterns

Use Case

- Medium- to long-horizon forecasting
- Detecting slow macroeconomic influence

3.3 Temporal Convolutional Networks (TCN)

Why TCNs

- Faster and more stable than LSTM
- Handles long sequences
- Parallelizable

What it provides

- Learns time dependencies without recurrence
- Robust to noisy indicators

Use Case

- High-frequency data (daily/weekly)
- Replacement or complement to LSTM

3.4 Transformer-Based Time Series Models (Advanced)

Examples

- Temporal Fusion Transformer (TFT)
- Informer

MACROSIGHT ROADMAP

- Autoformer

Why this matters

- Learns which indicators matter at which time
- Handles multiple horizons simultaneously
- Built-in attention (interpretability)

Critical Advantage

- Automatically weights macro indicators dynamically
- Adapts to regime changes

Use Case

- Executive-grade forecasts
- Scenario simulations

---

4. Online & Incremental Learning (Continuous Adaptation)

To meet *"learns as new data comes"*:

4.1 Incremental Learning Models

Examples

- Online Gradient Boosting
- River (formerly Creme) library
- SGD-based regressors

Behavior

- Model updates weights per batch
- No full retraining required
- Adapts to data drift

Use Case

- Near real-time updates
- High-velocity indicators (FX, fuel)

4.2 Periodic Retraining Strategy (Recommended)

In enterprise systems, **pure online learning is risky**.

MacroSight™ should use:

- Incremental updates daily

MACROSIGHT ROADMAP

- Full retraining weekly/monthly
- Governance checkpoints per retrain

---

5. Feature Learning (No Explicit Programming)

To avoid manual feature engineering:

5.1 Automatic Lag Learning

- LSTM / TCN / Transformers learn lags implicitly
- No hardcoded "last 3 months" logic

5.2 Embeddings for Categorical & Policy Data

- Product embeddings
- Policy-event embeddings
- Region embeddings

These are **learned representations**, not enums.

---

6. Ensemble Learning (Enterprise Standard)

MacroSight™ should **never rely on a single model**.

Recommended Ensemble

Model Type	Role
GBDT	Strong baseline
LSTM / TCN	Temporal depth
Transformer	Indicator attention
Linear baseline	Sanity check

Final forecast:

Forecast = weighted ensemble(predictions)

Weights are learned from validation data.

---

7. Concept Drift Detection (Critical)

Macro data shifts. Models must adapt.

Techniques

- Population Stability Index (PSI)

MACROSIGHT ROADMAP

- ADWIN drift detection
- Rolling error monitoring

Outcome

- Trigger retraining
- Flag regime change
- Preserve auditability

---

8. Explainability (Non-Negotiable)

Enterprise forecasting requires **trust**.

Mandatory Techniques

- SHAP values (GBDT, neural models)
- Attention visualization (Transformers)
- Feature contribution reports

Executives must see:

Why the model predicts what it predicts.

---

9. Governance Layer (Backend-Controlled)

Every model must log:

- Training data window
- Indicators used
- Hyperparameters
- Accuracy metrics
- Drift status

This aligns with your **ModelRun** domain entity.

---

10. Recommended ML Stack (Colab Training)

Layer	Technology
Data	Pandas, NumPy
Models	XGBoost, PyTorch
Transformers	PyTorch Forecasting

MACROSIGHT ROADMAP

Layer	Technology
Drift	River
Explainability	SHAP
Serialization	joblib / ONNX
Versioning	Model registry (backend)

MACROSIGHT ROADMAP